

Name: Kanika Saini

Program: Btech CSE B16

SAP ID: 590034098

HDOC Day-28– Arrays in C

Problem Statement

Write a **menu-driven C program** to perform the following operations:

1. Count even, odd, positive, negative, and zero elements in an array.
2. Search for an element using **Linear Search**.
3. Merge two arrays.
4. Exit.

1. Algorithm

Step 1: Start.

Step 2: Display the menu:

1. Count even, odd, positive, negative and zero elements
2. Linear Search
3. Merge two arrays
4. Exit

Step 3: Read the user's choice.

Step 4: Use a `switch` statement to perform the selected operation.

Case 1: Count Elements

1. Input the size `n` of the array.
2. Input `n` elements.

3. Initialize:
even = 0, odd = 0, positive = 0, negative = 0, zero = 0.
4. Traverse the array.
5. For every element:
 - o If it is 0, increment zero.
 - o Else if it is positive, increment positive.
 - o Else increment negative.
 - o If it is divisible by 2, increment even.
 - o Otherwise increment odd.
6. Display all counts.

Case 2: Linear Search

1. Input the size `n`.
2. Input the array elements.
3. Input the element to search, `key`.
4. Traverse the array from beginning to end.
5. Compare each element with `key`.
6. If found, display its position.
7. Otherwise display **Element not found**.

Case 3: Merge Two Arrays

1. Input size `n1` and elements of the first array.
2. Input size `n2` and elements of the second array.
3. Copy the first array into the merged array.
4. Copy the second array after the first array.
5. Display the merged array.

Case 4

Exit the program.

Step 5: If the choice is invalid, display an error message.

Step 6: Continue displaying the menu until the user selects 4.

Step 7: Stop.

2. Test Case Table

Test Case	Operation	Input	Expected Output	Actual Output	Result
1	Count elements	5: 2 -3 0 4 -7	Even = 3, Odd = 2, Positive = 2, Negative = 2, Zero = 1	Same as expected	Pass
2	Linear Search	Array: 10 20 30 40 50, Key: 30	Element found at position 3	Element found at position 3	Pass
3	Linear Search	Array: 2 4 6 8, Key: 5	Element not found	Element not found	Pass
4	Merge Arrays	A: 1 2 3, B: 4 5 6	1 2 3 4 5 6	1 2 3 4 5 6	Pass
5	Invalid menu	Choice = 7	Invalid choice	Invalid choice	Pass

Note: In Test Case 1, 0 is counted as **even and zero**, but it is neither positive nor negative.

3. C Program

```
#include <stdio.h>

int main()
{
    int choice;
    int a[100], b[100], merged[200];
    int n, n1, n2;
    int i, key, found;
    int even, odd, positive, negative, zero;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Count even, odd, positive, negative and zero elements\n");
        printf("2. Linear Search\n");
        printf("3. Merge two arrays\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter size of array: ");
                scanf("%d", &n);

                printf("Enter %d elements: ", n);
                for(i = 0; i < n; i++)
                {
```

```

        scanf("%d", &a[i]);
    }

    even = 0;
    odd = 0;
    positive = 0;
    negative = 0;
    zero = 0;

    for(i = 0; i < n; i++)
    {
        /* Count even and odd */
        if(a[i] % 2 == 0)
            even++;
        else
            odd++;

        /* Count positive, negative and zero */
        if(a[i] > 0)
            positive++;
        else if(a[i] < 0)
            negative++;
        else
            zero++;
    }

    printf("\nEven elements      = %d\n", even);
    printf("Odd elements       = %d\n", odd);
    printf("Positive elements = %d\n", positive);
    printf("Negative elements = %d\n", negative);
    printf("Zero elements      = %d\n", zero);

    break;

case 2:
    printf("Enter size of array: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    found = 0;

    for(i = 0; i < n; i++)
    {
        if(a[i] == key)
        {
            printf("Element found at position %d\n", i + 1);
            found = 1;
            break;
        }
    }

```

```

    }

    if(found == 0)
    {
        printf("Element not found\n");
    }

    break;

case 3:
    printf("Enter size of first array: ");
    scanf("%d", &n1);

    printf("Enter %d elements of first array: ", n1);
    for(i = 0; i < n1; i++)
    {
        scanf("%d", &a[i]);
    }

    printf("Enter size of second array: ");
    scanf("%d", &n2);

    printf("Enter %d elements of second array: ", n2);
    for(i = 0; i < n2; i++)
    {
        scanf("%d", &b[i]);
    }

    /* Copy first array */
    for(i = 0; i < n1; i++)
    {
        merged[i] = a[i];
    }

    /* Copy second array */
    for(i = 0; i < n2; i++)
    {
        merged[n1 + i] = b[i];
    }

    printf("Merged array: ");

    for(i = 0; i < n1 + n2; i++)
    {
        printf("%d ", merged[i]);
    }

    printf("\n");

    break;

case 4:
    printf("Exiting program...\n");
    break;

default:
    printf("Invalid choice! Please enter 1 to 4.\n");

```

```
    }  
    } while(choice != 4);  
    return 0;  
}
```

4. Explanation of Important Logic

Operation 1 – Even/Odd

```
if(a[i] % 2 == 0)  
    even++;  
else  
    odd++;
```

The % operator gives the remainder. If the remainder after division by 2 is 0, the number is even.

For example:

```
8 % 2 = 0 → Even  
7 % 2 = 1 → Odd
```

Positive/Negative/Zero

```
if(a[i] > 0)  
    positive++;  
else if(a[i] < 0)  
    negative++;  
else  
    zero++;
```

5. Linear Search Logic

Linear search checks elements **one by one from the beginning**.

For:

```
Array = 10 20 30 40 50  
Key = 30
```

The comparisons are:

```
30 == 10 → No  
30 == 20 → No  
30 == 30 → Yes
```

Therefore:

Element found at position 3

The important code is:

```
for(i = 0; i < n; i++)
{
    if(a[i] == key)
    {
        printf("Element found at position %d\n", i + 1);
        found = 1;
        break;
    }
}
```

We print `i + 1` because array indexing starts from **0**, while normal positions start from **1**.

6. Merging Logic

Suppose:

First array:
1 2 3

Second array:
4 5 6

First copy `a[]`:

`merged[] = 1 2 3`

Then copy `b[]` starting from index `n1`:

`merged[] = 1 2 3 4 5 6`

So the final output is:

Merged array: 1 2 3 4 5 6

7. Sample Execution

Test case1-

```
(kali㉿kali)-[~/Desktop]
$ cc day28.c

(kali㉿kali)-[~/Desktop]
$ ./a.out

===== MENU =====
1. Count even, odd, positive, negative and zero elements
2. Linear Search
3. Merge two arrays
4. Exit
Enter your choice: 1
Enter size of array: 5
Enter 5 elements: 2 -3 0 4 -7

Even elements      = 3
Odd elements       = 2
Positive elements  = 2
Negative elements  = 2
Zero elements      = 1
```

Test case 2-

```
===== MENU =====
1. Count even, odd, positive, negative and zero elements
2. Linear Search
3. Merge two arrays
4. Exit
Enter your choice: 2
Enter size of array: 5
Enter 5 elements: 10 20 30 40 50
Enter element to search: 30
Element found at position 3
```

Test case 3-

```
===== MENU =====
1. Count even, odd, positive, negative and zero elements
2. Linear Search
3. Merge two arrays
4. Exit
Enter your choice: 2
Enter size of array: 4
Enter 4 elements: 2 4 6 8
Enter element to search: 5
Element not found
```

Test case 4-

```
===== MENU =====
1. Count even, odd, positive, negative and zero elements
2. Linear Search
3. Merge two arrays
4. Exit
Enter your choice: 3
Enter size of first array: 3
Enter 3 elements of first array: 1 2 3
Enter size of second array: 3
Enter 3 elements of second array: 4 5 6
Merged array: 1 2 3 4 5 6
```

Test case 5-

```
===== MENU =====
1. Count even, odd, positive, negative and zero elements
2. Linear Search
3. Merge two arrays
4. Exit
Enter your choice:
7
Invalid choice! Please enter 1 to 4.
```